



Automation Tool : Codex

Day 2 - AI for High-Value Food Processing & Operations



Agenda

1. What's codex ?
2. Installation
3. Tutorial
4. Useful Resources
5. Comparison
6. Extension: OpenAI Codex vs. Claude Code

What Is OpenAI Codex?

- Codex is an AI coding and automation agent that can **inspect project files, plan tasks, edit files, run commands, use tools, and produce working outputs under user's environments.**



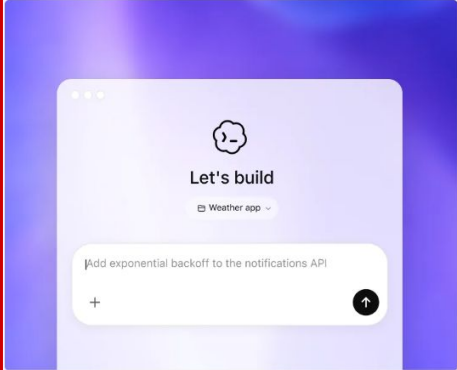
Codex - Installation

OpenAI Research Products Business Developers Company Foundation

Log in

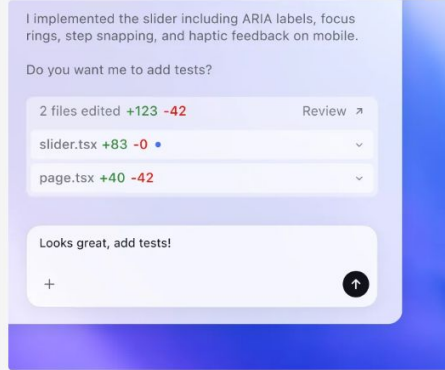
Try ChatGPT

We will use the desktop version to demo.



Start in the Codex app

Download for Windows



Move to your editor

Try in your IDE



Keep going in the terminal

```
$ npm i -g @openai/codex
```

Download: <https://developers.openai.com/codex/app>



Codex Tutorial

Prerequisite



There are three ways to access Codex:

1. Free Tier:

We estimate needing about **two** free-tier ChatGPT accounts to cover token usage for this demo.

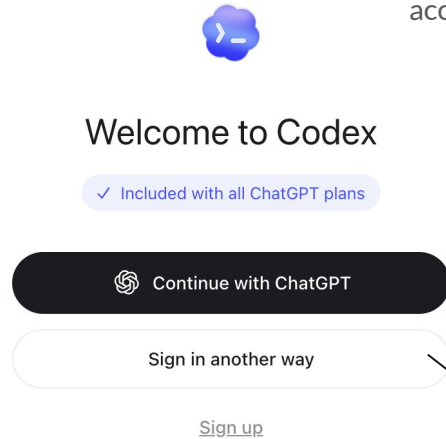
2. Paid Tier (ChatGPT Subscription):

Codex is included with all ChatGPT subscriptions.

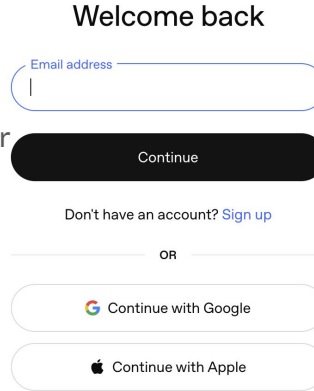
3. Pay-as-you-go (API Key):

Obtain an API Key through the OpenAI Platform : Manual

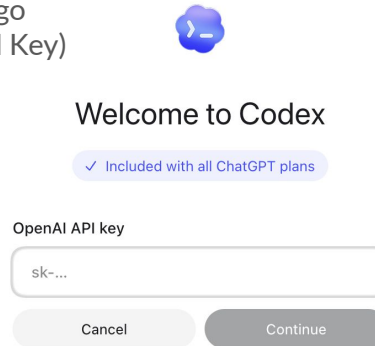
Prerequisite



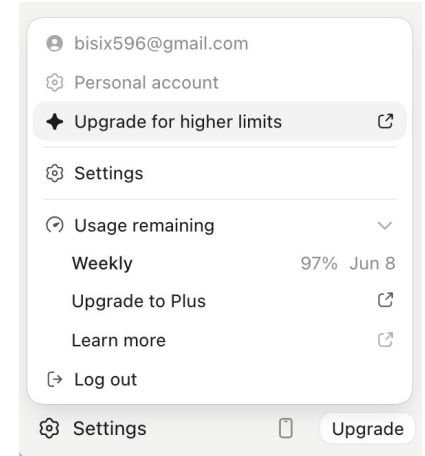
Way 1: Included in ChatGPT Subscription (also available for free-tier accounts)



Way 2: Pay-as-you-go (OpenAI API Key)

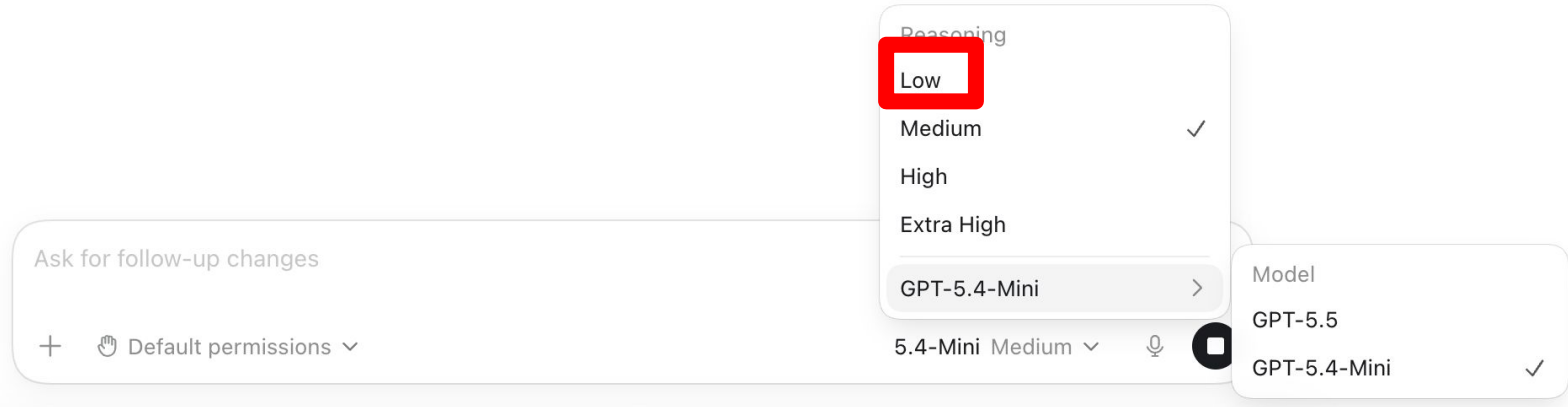


Check token usage limit here:



Check token usage via OpenAI Dashboard:
<https://platform.openai.com/settings/organization/usage>

Prerequisite



(Optional) Select GPT-5.4-mini with **Low** Reasoning to reduce your running time and token usage during the demo.

Demo Overview



1. Lab Sheet: [Codex_Workshop_Lab_Sheet_EN_refined.docx](#)
2. Please download and extract the required prerequisite files onto your laptop:
Attached Files: [codex_lab_attachments.zip](#)

Demo Overview



Lab	Topic	Learner outcome
1	Handle Local Files + Plan Mode	See the difference between acting and planning, use codex to handle your files
2	Plugins and Automation	Use simple plugin and automation
3	Reusable skill, Global SKILL.md	Turn a conversation into a repeatable skill
4	Global AGENTS.md	Apply behavior and coding rules across projects
5	Single agent project	Combine project rules, skills, data, and outputs
6	Sub-agents	Orchestrate specialized agents in stages
7	MCP Server	Familiar with Model Context Protocol to connect to Database Server (Context-7 as Database server for this case.)

Lab 1A: Handle Local Files + Plan Mode

Load prepared files:

Project

Start from scratch

Use an existing folder

codex_lab_attachment

codex_lab_attachment.zip

lab01_A_messy_files

lab01_B_messy_files

lab02_plugin_workspace

lab03_iterative_to_skill_project

lab04_no_file

lab5_single_agent

Choose folder

- lab01_A_messy_files for Lab 1A
- lab02_A_messy_files for Lab 1A

Then, follow the steps in lab sheet.

Task:

You are asked to handle a messy file directory containing duplicate files, backup files, temporary files, and old files.

lab01_A_messy_files

Name

- deviation_export_copy.csv
- deviation_export_final.csv
- old_report.md
- qa_note.txt
- report_template_old.md
- risk_rule_draft.txt

Lab 1A: Handle Local Files

lab01_A_messy_files	
Name	
	deviation_export_copy.csv
	deviation_export_final.csv
	old_report.md
	qa_note.txt
	report_template_old.md
	risk_rule_draft.txt

I reorganized the folder into a clean Food QA analysis project.

- Canonical raw data is now at  [deviation_export_final.csv](#)
- The duplicate export is archived at  [deviation_export_copy.csv](#)
- Draft notes are in  [qa_note.txt](#) and  [risk_rule_draft.txt](#)
- Old report material is in  [old_report.md](#)
- The report template is in  [report_template_old.md](#)
- I added a project guide at  [README.md](#)

Name	
>	 data
>	 notes
	 README.md
>	 reports

name	
	qa_note.txt
	risk_rule_draft.txt

Lab 1B: Handle Local Files + Plan Mode

Plan mode Turn plan mode off

/plan

+ Default permissions Plan

Do anything

+ Default permissions Plan

Listed files, ran 1 command

I've confirmed this is a small bundle of a few CSVs plus draft notes and reports. Next I'm checking the file contents so I can separate raw inputs from drafts and figure out the clean project layout.

Explored 6 files

I've got the raw material now: two copies of the same deviation export, a draft severity scale, and two incomplete report notes. I'm checking the data for the kinds of cleanup we should preserve in the final project structure so the organization reflects the real analysis needs, not just file names.

Ran 4 commands

I've confirmed the final CSV has a few normalization issues that belong in the analysis layer: one duplicate batch, mixed severity casing, one blank severity, and inconsistent date formats. I'm comparing the two exports now so we can keep the right one as the canonical dataset and move the other out of the active analysis path.

Ran 3 commands

The two CSVs are not identical, so one of them should become the raw snapshot and the other the working dataset, not both sitting loose in the root. I'm taking one quick look for hidden or scaffold files so I don't miss an existing project convention before I draft the organization plan.

Plan

Food QA Analysis Project Cleanup

Summary

Turn the loose files in the folder into a small, traceable Food QA analysis project with clear separation between raw data, working data, notes, and report artifacts.

Key Changes

- Create a simple project layout such as:
 - data/raw/ for source exports
 - data/processed/ for cleaned or normalized analysis-ready data
 - notes/ for QA observations and rule drafts
 - reports/ for report drafts and templates
 - README.md for project purpose, file map, and how to use the data
- Treat deviation_export_final.csv as the canonical raw export and keep deviation_export_copy.csv as a raw archive duplicate snapshot for traceability.
- Preserve the raw CSVs unchanged, and place any normalized version in data/processed/

Implement this plan?

1. Yes, implement this plan

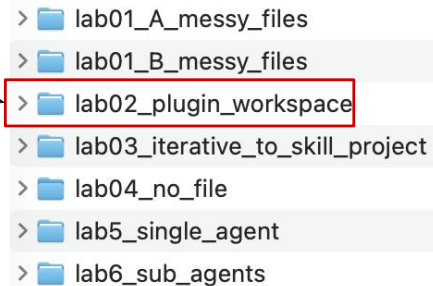
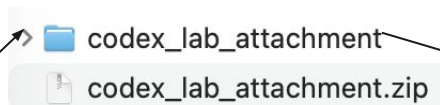
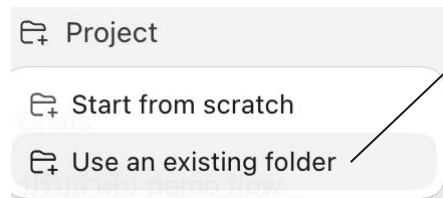
2. No, and tell Codex what to do differently

Dismiss ESC Submit ↩

13

Lab 2: Plugins and Automation

Load prepared files:



New chat

Search

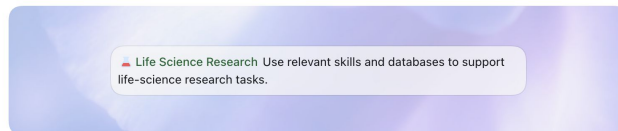
Plugins

Automations



Life Science Research

General life-sciences research with routing, evidence synthesis, and optional parallel subagent analysis



Internal life-science research workflows that help Codex interpret a user's research question, normalize the relevant entities, choose the right skills, and synthesize evidence-backed answers across public resources. The plugin spans human genetics, functional genomics, expression, pathways, protein structure, chemistry, pharmacology, literature, clinical evidence, and public study discovery, with a research-router entrypoint for broad tasks and optional subagent-assisted parallel work when evidence lanes are independent.

Lab 2: Plugins and Automation

 **Life Science Research** Investigate *Lactobacillus plantarum* for food fermentation. Give a short, business-friendly summary of:

- what it is
 - why it matters in food processing
 - key traits for acid tolerance and flavor production
 - any important strain differences or risks
 - practical implications for dairy and vegetable fermentation
- End with 2–3 concise recommendations for process teams.

3:12 PM 

Working for 11s

I'm using the Life Science Research route to pull a compact, evidence-based view of *Lactobacillus plantarum* for food fermentation, then I'll turn that into a business-friendly summary and process recommendations.

 Explored 1 file

I'm narrowing this to literature evidence: taxonomy/identity, fermentation traits, and practical strain-level differences. That should keep the summary defensible and useful for process teams.

 Reading Ncbi Entrez Skill skill

Working for 29s

I'm using the Life Science Research route to pull a compact, evidence-based view of *Lactobacillus plantarum* for food fermentation, then I'll turn that into a business-friendly summary and process recommendations.

 Explored 1 file

I'm narrowing this to literature evidence: taxonomy/identity, fermentation traits, and practical strain-level differences. That should keep the summary defensible and useful for process teams.

 Searching the web for Lactiplantibacillus plantarum acid tolerance genes review | pmc.ncbi.nlm.nih.gov

 Read Ncbi Entrez Skill skill

Ran command

```
Shell
$ echo '{"endpoint":"esearch","params":{"db":"pubmed","term":"Lactobacillus plantarum vegetable fermentation review[Title/Abstract]","retmode":"json","retmax":5},"max_items":5}' | python /Users/prommin/.codex/plugins/cache/openai-curated/life-science-research/8770e9d2/skills/ncbi-entrez-skill/scripts/ncbi_entrez.py
{"ok": false, "error": {"code": "network_error", "message": "Request failed: HTTPSConnectionP
```

Exit code 1

Ran command

-  Lactiplantibacillus plantarum fermentation acid t
-  Lactiplantibacillus plantarum acid tolerance gene
-  Lactiplantibacillus plantarum biogenic amines sa
-  <https://pmc.ncbi.nlm.nih.gov/articles/PMC90261>

Lab 2: Plugins and Automation

Worked for 49s >

Short take

Lactiplantibacillus plantarum is the current name for what many teams still call *Lactobacillus plantarum*. It is a highly adaptable lactic acid bacterium that is widely used in food fermentation because it tolerates harsh processing conditions, acidifies products quickly, and can improve aroma, texture, and shelf life. It is especially relevant in plant fermentations, but some strains also perform well in dairy as adjunct cultures.

Sources: [review](#), [strain diversity review](#)

Why it matters in food processing

- It is a “generalist” fermenter that shows up in vegetables, dairy, sourdough, and other food matrices.
- It helps drive fast acidification, which improves microbial stability and reduces spoilage risk.
- It can contribute desirable flavor compounds and sometimes bioactive compounds that add product value.
- It is often useful as a starter or co-culture where a reliable, robust LAB is needed.

Sources: [review](#), [carbohydrate metabolism review](#)

Key traits for acid tolerance and flavor

- Acid tolerance comes from stress-response systems such as proton pumping, membrane remodeling, and inducible acid-stress genes.
- Some strains show an “acid tolerance response” after prior mild acid exposure.
- Flavor contribution is strain-dependent and can come from carbohydrate utilization, proteolysis/lipolysis support, and formation of acids and aroma-active metabolites.
- Reported flavor-related outputs include improved milk/vegetable flavor, acetic acid, and 2-butanone in some co-culture systems.

Sources: [acid stress review/article](#), [acid stress mechanisms](#), [flavor-related enzyme profile](#), [dairy flavor note](#)

please create the final report into markdown file in reports/.

Working for 13s

I'm turning the summary into a clean report file under `reports/` and will first check the workspace layout so I place it in the right spot.

 Listed files, ran 1 command

I found an empty `reports/` folder, so I'm adding a single markdown report there with the business-friendly summary and recommendations.

 Creating `lactiplantibacillus_plantarum_fermentation_report.md` +52 -0

Lab 2: Plugins and Automation





Files
Browse project files



Side chat
Start a side conversation



Browser
Open a website



Terminal
Start an interactive shell

lactiplantibacillu... +

lab02_plugin_workspace > reports > lactiplantibacillus_plantarum_fermentation_report.md

Lactiplantibacillus plantarum in Food Fermentation

What it is

Lactiplantibacillus plantarum is the current name for the bacterium many teams still call *Lactobacillus plantarum*. It is a highly adaptable lactic acid bacterium commonly found in fermented foods, especially vegetables, but also dairy, sourdough, and other food matrices.

Why it matters in food processing

This species is valued because it can:

- acidify products quickly
- help stabilize fermentation and suppress spoilage organisms
- improve texture, shelf life, and sensory quality
- support production of desirable flavor compounds in some strains

In practice, it is often used as a starter or adjunct culture when processors want a robust, flexible organism that performs well across different food systems.

Key traits for acid tolerance and flavor production

Its industrial usefulness comes from strong stress adaptation and broad carbohydrate metabolism.

- **Acid tolerance:** Many strains survive low pH through proton-pumping systems, membrane remodeling, and acid-stress response genes.
- **Stress adaptation:** Some strains become more acid-tolerant after prior mild acid exposure.
- **Flavor formation:** Strains differ in their ability to release or transform sugars, amino acids, and lipids into acids and aroma-active compounds.
- **Functional outputs:** Depending on the strain and matrix, it can contribute lactic acid, acetic acid, 2-butanone, exopolysaccharides, and other metabolites tied to flavor and mouthfeel.

Important strain differences and risks

Performance is highly strain-specific. Two strains with the same species name can behave very differently in acid tolerance, flavor generation, and safety profile.

Key risks to screen for:

- transferable antibiotic resistance genes
- mobile genetic elements
- virulence factors
- biogenic amine production

Filter files...

reports

lactiplantibacillus_plantimd

Lab 2: Plugins and Automation

 **Life Science Research** Research interesting and relevant studies that can support food quality and process improvement work. Every morning, provide a concise summary of key trends, important findings, and practical implications. The report should not exceed one A4 page in length. Save the report in the reports/ directory using the following filename format: YYYYMMDD_daily_report.md



Working for 19s

I'm setting up a daily workspace automation so the report is generated each morning and saved into reports/ with the date-based filename you specified.

The first automation call was rejected by the app schema, so I'm retrying with the leanest valid fields to get the schedule in place.

The app still isn't accepting a direct create payload, so I'm switching to a proposal-style automation call to see the exact schema it wants. ⌚ Created · [Daily food science report](#) · Daily at 8:28 AM

Daily food science report

Research recent and interesting scientific studies relevant to food quality and process improvement work, with emphasis on practical implications for fermentation and related food processing work. Produce a concise markdown report no longer than one A4 page. Summarize only the key trends, important findings, and practical implications. Save the report in the reports directory using the filename format YYYYMMDD_daily_report.md.

Status

Status

Next run

Last ran

● Active

Tomorrow at 8:29 AM

-

Details

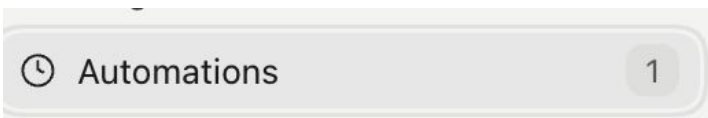
Chat

Assess Lactobacillus plantarum

Interval

Daily at 8:28 AM

Lab 2: Plugins and Automation



Automations

Current

☐ Daily food science report Heartbeat • Assess Lactob...

Daily at 8:29 AM

Don't forget to delete this automation to save your token usage.

Test by

- Click Run now or
- Change to smaller interval

Interval

Every 2m ▾

Schedule

Interval ▾

Every minutes

⏸️ 🗑️ ▶ Run now

Status

Status

● Active

Next run

Tomorrow at 8:29 AM

Last ran

-

Details

Runs in

Chat ▾

Chat

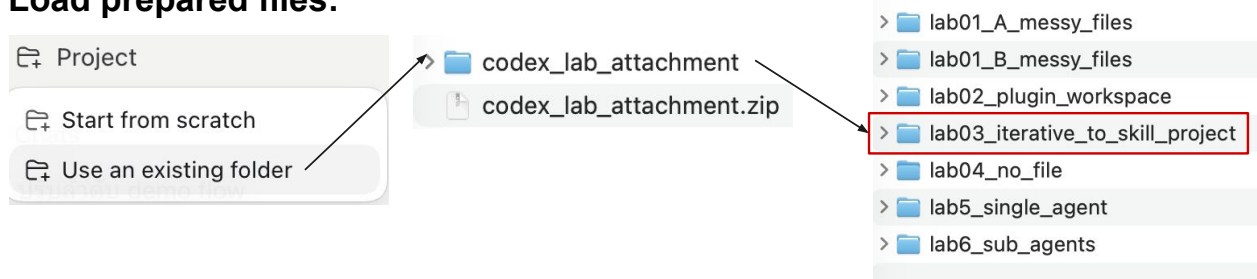
Assess Lactobacillus plantarum ▾

Interval

Daily at 8:29 AM ▾

Lab 3: Create Your Own Reusable Skills

Load prepared files:



Analyze data/raw/food_deviation_log.csv and create a QA report as a Word document using [Documents](#)

Create:
reports/risk_report.docx

Focus on the main food safety and quality risks.

Do not modify files under data/raw/.

Do not use LibreOffice for validation; keep the final DOCX and explain the limitation briefly.

Lab 3: Create Your Own Reusable Skills

A	B	C	D	E	F	G	H	I	J	K	L	
date	product	process_step	issue_type	severity	root_cause	action_taken	downtime	batch_id	detected_by	facility	shift	
13/5/2026	Ready-to-eat	Washing	Chlorine level	High	Sanitizer dos	Adjusted dos	32	B-2001	QA Inspector	Plant A	Morning	
13/5/2026	Frozen Dumpling	Cooking	Core temper	Critical	Steam press	Extended cook	88	B-2002	QA Inspector	Plant C	Evening	
14/5/2026	Cold-pressed	Filling	Underfilled	Medium	Filler head	Not cleaned	24	B-2003	Line Leader	Plant A	Morning	
14/5/2026	Fresh-cut	Meat Cutting	Size inconsis	Low	New blade	Good	Adjusted	12	B-2004	Production S	Plant B	Evening
15/5/2026	Ready-to-eat	Storage	Temperature	Critical	Cold room	door	Moved	82	B-2005	QA Manager	Plant A	Night
15/5/2026	Frozen Dumpling	Mixing	Incorrect ing	Critical	Scale calibra	Stopped	batch	95	B-2006	Production S	Plant C	Morning
15/5/2026	Frozen Dumpling	Mixing	Incorrect ing	Critical	Scale calibra	Stopped	batch	95	B-2006	Production S	Plant C	Morning

Ran 8 commands

I've got enough analysis to draft the report. I'm moving to document creation now, and I'll keep the structure tight: executive summary, key risk themes, detailed observations, and immediate actions, with a small note on log-quality issues because they matter for QA credibility.

Explored 2 files, ran 3 commands

I've got the risk story now: this log is dominated by cold-chain failures, formulation control problems, and packaging/traceability issues. I'm turning that into a report with a small metric strip and a ranked risk table so the biggest food-safety impacts are obvious at a glance.

I'm writing the generator now. It will read the CSV, normalize the messy log entries for analysis, and emit the finished DOCX into `reports/risk_report.docx` with tables sized explicitly so the report stays readable.

Creating `build_risk_report.py +273 -0`



risk_report.docx
Document · DOCX

Food Deviation QA Risk Report

Focused review of food safety and quality deviations recorded in May 2026

Raw rows	Unique incidents	High/Critical incidents	Unique downtime
24	23	13	944 min

Source: data/raw/food_deviation_log.csv

Scope note: The analysis below deduplicates one exact duplicate record and normalizes inconsistent severity labels for reporting.

Executive Summary

The log contains 23 unique incidents (from 24 raw rows) and 13 high-or-critical events. Cold-chain control is the dominant risk theme, accounting for 5 incidents and 330 minutes of downtime.

Plant A has the highest incident volume, while Plant C concentrates the most severe events because of formulation, thermal-processing, and contamination failures in frozen dumpling production.

Packaging and traceability controls are also recurring weak points, with label mismatch, seal defects, and cap issues creating avoidable release and recall risk.

Top Risk Themes

Table 1. Ranked risk themes based on unique incidents, severity, and downtime.

Risk theme	Evidence	Why it matters	Immediate focus
Cold-chain control	5 incidents / 330 min	Repeated temperature abuse in storage and transfer raises pathogen growth and shelf-life risk.	Tighten transfer checks, alarm response, and cold-room verification.
Recipe / formulation control	2 incidents / 155 min	Recipe version drift and missing ingredients can change product composition and regulatory status.	Lock version control and require QA sign-off before mixing.
Contamination control	3 incidents / 123 min	Foreign material and metal detector rejection create direct physical contamination and recall exposure.	Strengthen raw screening, equipment inspection, and challenge checks.
Packaging / traceability	3 incidents / 97 min	Label mismatch and line-clearance failures can create mislabeling and traceability errors.	Enforce pre-start label verification and line clearance.
Thermal processing	1 incident / 85 min	Low core temperature indicates an underprocessed product that may not be safe to release.	Review cook validation and steam-pressure monitoring.

Where the Risk Concentrates

Plant A records the most incidents (12) and is dominated by cold-chain, foreign-material, and fill-control issues in salad and juice lines.

Plant B shows mostly packaging and sanitation drift, including size inconsistency, underfill, seal defects, and one chlorine-level excursion.

Plant C has the highest-severity mix: recipe ratio errors, missing ingredient risk, metal-detection rejection, and an under-temperature cook event.

Immediate Actions

Table 2. Priority actions to reduce the highest-probability and highest-severity risks.

Priority	Action	Primary risk addressed
1	Lock cold-room checks at every transfer and require documented alarm response within the shift.	Temperature abuse
2	Make recipe version control and ingredient reconciliation mandatory before mixing starts.	Incorrect ingredient ratio / missing ingredient
3	Strengthen metal-detector checks, equipment wear inspection, and raw-material screening.	Foreign material / metal contamination
4	Require line clearance and label verification before start-up and after every changeover.	Label mismatch / traceability
5	Standardize the log fields: one severity taxonomy, one date format, and mandatory root cause entry.	Reporting quality

Data Quality Notes

Duplicate entry: One exact duplicate row appears for the 2026-05-03 Frozen Dumpling / Incorrect ingredient ratio event; the report analyzes unique incidents to avoid double counting.

Severity coding: Severity values are inconsistent in the source file: one row is blank and one uses lower-case 'critical'.

Date format: The log mixes date styles such as 2026-05-04, 2026/05/04, and 05/05/2026; this should be standardized for downstream QA tracking.

Missing root cause: The 'Missing ingredient' record has an empty root-cause field, which weakens corrective-action follow-up.

Lab 3: Create Your Own Reusable Skills

Read reports/risk_report.docx and data/raw/food_deviation_log.csv.

Create presentation-ready PNG plots under reports/figures/ and insert them in appropriate places in the existing Word report:

1. Downtime minutes by issue type, sorted descending.
2. Deviation count by severity, normalizing severity case where appropriate.

Use readable labels, clean styling, sensible colors, titles, and short captions explaining each key insight.

Do not modify files under data/raw/.

Do not use LibreOffice for validation.

I've confirmed the report structure, so I'm adding the visuals in a way that keeps the chart for downtime concentration and one for severity distribution, each with its own caption saved under reports/figures/.



risk_report.docx

Document · DOCX



downtime_by_issue_type.png

Image · PNG



deviation_count_by_severity.png

Image · PNG

```
build_risk_report.py X
Users > prominin > Downloads > codex_lab_attachment > lab03_iterative_to_skill_project > build_risk_report
1 from __future__ import annotations
2
3 import csv
4 from collections import Counter, defaultdict
5 from datetime import date
6 from pathlib import Path
7
8 import matplotlib.pyplot as plt
9 from docx import Document
10 from docx.enum.section import WD_SECTION
11 from docx.enum.table import WD_TABLE_ALIGNMENT, WD_CELL_VERTICAL_ALIGNMENT
12 from docx.enum.text import WD_ALIGN_PARAGRAPH
13 from docx.oxml import OxmlElement
14 from docx.oxml.ns import qn
15 from docx.shared import Inches, Pt, RGBColor
16
17
18 ROOT = Path(__file__).resolve().parent
19 CSV_PATH = ROOT / "data" / "raw" / "food_deviation_log.csv"
20 OUT_PATH = ROOT / "reports" / "risk_report.docx"
21 FIG_DIR = ROOT / "reports" / "figures"
22
23
24 def set_run_font(run, name="Calibri", size=None, color=None, bold=None, italic=None):
25     run.font.name = name
26     rpr = run._element.get_or_add_rPr()
27     rfonts = rpr.rFonts
28     rfonts.set(qn("w:ascii"), name)
29     rfonts.set(qn("w:hAnsi"), name)
30     if size is not None:
31         run.font.size = Pt(size)
32     if color is not None:
33         if isinstance(color, RGBColor):
34             run.font.color.rgb = color
35         else:
36             run.font.color.rgb = RGBColor.from_string(color.replace("#", ""))
37     if bold is not None:
38         run.bold = bold
39     if italic is not None:
```

risk_report.docx +			
risk_report DOC			
< 1/1 > 75%			
Contamination control	3 incidents / 123 min	foreign material and metal detector rejection create direct physical contamination and recall exposure.	strengthen raw screening, equipment inspection, and challenge checks.
Packaging / traceability	3 incidents / 97 min	Label mismatch and line-clearance failures can create mislabeling and traceability errors.	Enforce pre-start label verification and line clearance.
Thermal processing	1 incident / 85 min	Low core temperature indicates an underprocessed product that may not be safe to release.	Review cook validation and steam-pressure monitoring.

Figures

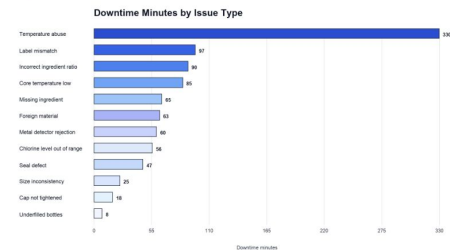


Figure 1. Downtime is concentrated in temperature abuse and recipe control issues, which create the largest operational and food-safety impact.

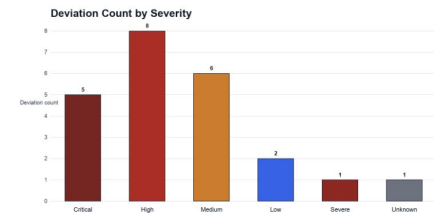


Figure 2. Most deviations are High or Medium severity, but the Critical and Severe events deserve disproportionate attention because they map to the highest food-safety risk.

Where the Risk Concentrates

Plant A records the most incidents (12) and is dominated by cold-chain, foreign-material, and fill-control issues in salad and juice lines.

Plant B shows mostly packaging and sanitation drift, including size inconsistency, underfill, seal defects, and one chlorine-level excursion.

Plant C has the highest-severity mix: recipe ratio errors, missing ingredient risk, metal-detection rejection, and an under-temperature cook event.

Immediate Actions

Table 2. Priority actions to reduce the highest-probability and highest-severity risks.

Lab 3: Create Your Own Reusable Skills

This output is good. Turn this workflow into a reusable Codex skill template named qa-report-w-visualize.

Create a reusable Codex skill in the user's installed skill directory so it can be discovered in future sessions.

```
python - <<'PY'
from pathlib import Path
base = Path.home() / '.codex' / 'skills' / 'qa-report-w-visualize'...
```

Expand

Click `Yes` so you can call this skill `everywhere` even outside this project directory.

1. Yes

2. Yes, and don't ask again this session

3. No, and tell Codex what to do differently

↑ ↓

Skip

Submit ↩



SKILL.md
Document · MD

Finally, SKILL.md will be provided as reusable skill for any agents.

SKILL.md +

Users > prominin > .codex > skills > qa-report-w-visualize > SKILL.md

Metadata

name	qa-report-w-visualize
description	Create food-safety or quality QA report DOCX files from CSV deviation logs, add presentation-ready charts, and place them into an existing or new Word report.

QA Report with Visualizations

Use this skill when the user wants a QA or risk report built from a deviation log, especially when the output should include presentation-ready charts inside a Word document.

Core workflow

1. Read the source CSV and the current DOCX, if one exists.
2. Normalize obvious reporting issues for analysis only, such as duplicate rows, inconsistent severity casing, and mixed date formats. Do not modify the raw data file.
3. Identify the main safety and quality risks using counts, severity, and downtime.
4. Create clean PNG figures under `reports/figures/` with readable labels, titles, and short insight captions.
5. Insert the figures into the report near the relevant analysis sections.
6. Keep the report concise and decision-oriented: executive summary, ranked risks, figures, actions, and a short data-quality note.
7. Verify the final DOCX structurally. Do not use LibreOffice for validation.

Default analysis pattern

When the log has fields like `issue_type`, `severity`, and `downtime_min`:

- Plot downtime by issue type, sorted descending.
- Plot deviation count by severity, normalizing severity case where needed.
- Call out the highest-risk themes first, not the most frequent minor defects.
- Mention data-quality issues only if they affect the trustworthiness of the analysis.

Chart guidance

- Use horizontal bars for long issue names and vertical bars for simple categorical counts.
- Use restrained, high-contrast colors and avoid cluttered legends.
- Label bars directly when possible.
- Keep figure captions short and insight-led.
- Save figures as PNGs in `reports/figures/` with stable filenames.

Report guidance

- Preserve an existing report when present; insert figures into the most relevant section.
- If creating a new report, keep the structure simple: title, summary metrics, executive summary, ranked risks, figures, actions, and data-quality notes.
- Do not edit files under `data/raw/`.
- Prefer concise prose over dense tables unless the task truly needs tabular comparison.

Practical checks

- Confirm the DOCX contains the inserted images.
- Confirm chart captions match the insight shown in the figure.
- If rendering or visual QA is unavailable, state that clearly rather than claiming validation.

Lab 3: Create Your Own Reusable Skills

SKILL.md +

Users > prominin > .codex > skills > qa-report-w-visualize > SKILL.md

```
Metadata
name      qa-report-w-visualize
description Create food-safety or quality QA report DOCX files from CSV deviation logs, add presentation-ready charts, and place them into an existing or new Word report.
```

QA Report with Visualizations

Use this skill when the user wants a QA or risk report built from a deviation log, especially when the output should include presentation-ready charts inside a Word document.

Core workflow

1. Read the source CSV and the current DOCX, if one exists.
2. Normalize obvious reporting issues for analysis only, such as duplicate rows, inconsistent severity casing, and mixed date formats. Do not modify the raw data file.
3. Identify the main safety and quality risks using counts, severity, and downtime.
4. Create clean PNG figures under `reports/figures/` with readable labels, titles, and short insight captions.
5. Insert the figures into the report near the relevant analysis sections.
6. Keep the report concise and decision-oriented: executive summary, ranked risks, figures, actions, and a short data-quality note.
7. Verify the final DOCX structurally. Do not use LibreOffice for validation.

Default analysis pattern

When the log has fields like `issue_type`, `severity`, and `downtime_min`:

- Plot downtime by issue type, sorted descending.
- Plot deviation count by severity, normalizing severity case where needed.
- Call out the highest-risk themes first, not the most frequent minor defects.
- Mention data-quality issues only if they affect the trustworthiness of the analysis.

Chart guidance

- Use horizontal bars for long issue names and vertical bars for simple categorical counts.
- Use restrained, high-contrast colors and avoid cluttered legends.
- Label bars directly when possible.
- Keep figure captions short and insight-led.
- Save figures as PNGs in `reports/figures/` with stable filenames.

Report guidance

- Preserve an existing report when present; insert figures into the most relevant section.
- If creating a new report, keep the structure simple: title, summary metrics, executive summary, ranked risks, figures, actions, and data-quality notes.
- Do not edit files under `data/raw/`.
- Prefer concise prose over dense tables unless the task truly needs tabular comparison.

Practical checks

- Confirm the DOCX contains the inserted images.
- Confirm chart captions match the insight shown in the figure.
- If rendering or visual QA is unavailable, state that clearly rather than claiming validation.

Chats >

...



Qa Report W Visualize Create food-safety or quality QA report DOCX files from CSV deviation logs, add pres...

Use @qa-report with the uploaded for_global_test.csv and create a new QA risk report.

+ Default permissions v

5.4-Mini Medium v



Use created skill to automate your work : Read .csv -> Report .docx -> Visualize

QA Risk Report

Food deviation log analysis for food_deviation_log_v2.csv

Source window: 2026-05-13 to 2026-05-26 | Generated on 2026-06-01

Summary metrics

Unique deviations	Total downtime
27	1316 min
Critical + High	Facilities impacted
17 records	3

Executive summary

- 1 The log contains 27 unique deviations after removing one exact duplicate row. Critical and High events make up 17 of 27 records (63%), so the risk profile is concentrated in higher-severity issues.
- 1 Temperature abuse is the single largest risk theme at 6 events and 428 minutes of downtime, representing about 33% of all lost time.
- 1 Cold-chain, cooking, and recipe-control failures are the biggest operational drivers, while labeling and metal-detection deviations add traceability and contamination-control risk.

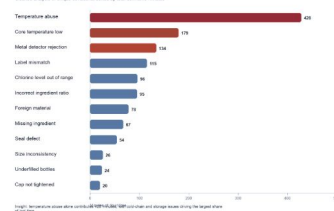
Ranked risks

- 1 1. Temperature abuse - 428 min across 6 event(s); severity mix: Critical 4, Severe 1, High 1.
- 2 2. Core temperature low - 179 min across 2 event(s); severity mix: Critical 2.
- 3 3. Metal detector rejection - 134 min across 2 event(s); severity mix: High 2.
- 4 4. Label mismatch - 115 min across 3 event(s); severity mix: High 2, Medium 1.
- 5 5. Chlorine level out of range - 96 min across 3 event(s); severity mix: High 2, Unspecified 1.

Figures

Downtime by issue type

Horizontal analysis of unique deviations, ranked by total downtime minutes

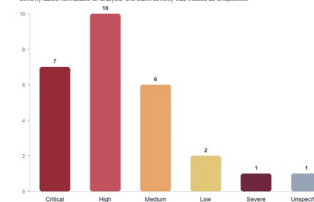


Insight: Temperature abuse alone accounts for 33% of total downtime, indicating a significant focus area for food safety and operational issues.

Figure 1. Downtime by issue type

Deviation count by severity

Severity labels normalized for analysis; one event severity was treated as Unspecified



Insight: Critical and High severities account for 17 of 27 unique deviations, so the risk profile is skewed toward food safety and compliance issues.

Figure 2. Deviation count by severity

Lab 4: Global AGENTS.md - Before



Edited mock_api_analysis_tool.py

+735 -0

Details

Before adjusting,

Codex agent usually generate all codes in one file (≥ 700 lines for this case).

We will change this behavior by AGENTS.md.

Here is the full code:

```
python

#!/usr/bin/env python3
"""Mock public API data analysis tool.

This script simulates a flaky paginated API, then fetches, validates, cleans,
analyzes, and reports on the returned records.
"""

from __future__ import annotations

import argparse
import csv
import dataclasses
import datetime as dt
import hashlib
import json
import logging
import math
import os
import random
import statistics
import sys
import textwrap
import time
from collections import Counter, defaultdict
from pathlib import Path
from typing import Any, Dict, Iterable, List, Optional, Sequence, Tuple
```

1 file changed +735 -0

[Review here](#)

```
@dataclasses.dataclass
class FetchStats:
    pages_requested: int = 0
    pages_succeeded: int = 0
    retries: int = 0
    rate_limits: int = 0
    failed_requests: int = 0
    raw_records_seen: int = 0

def setup_logging(output_dir: Path) -> logging.Logger:
    output_dir.mkdir(parents=True, exist_ok=True)
    logger = logging.getLogger("mock_api_tool")
    logger.setLevel(logging.INFO)
    logger.handlers.clear()

    fmt = logging.Formatter("%(asctime)s | %(levelname)s | %(message)s")
    console = logging.StreamHandler(sys.stdout)
    console.setFormatter(fmt)
    file_handler = logging.FileHandler(output_dir / "run.log", encoding="utf-8")
    file_handler.setFormatter(fmt)

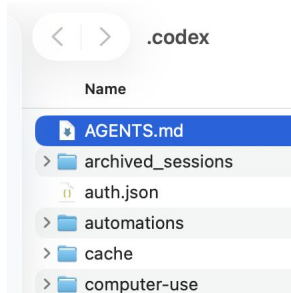
    logger.addHandler(console)
    logger.addHandler(file_handler)
    return logger

def parse_date(value: str) -> dt.date:
    for fmt in DATE_FORMATS:
        try:
            parsed = dt.datetime.strptime(value, fmt)
            return parsed.date()
        except ValueError:
            pass
    raise ValueError(f"Unrecognized date format: {value!r}")

def parse_datetime(value: str) -> dt.datetime:
    for fmt in DATE_FORMATS:
        try:
            return dt.datetime.strptime(value, fmt)
        except ValueError:
            pass
    raise ValueError(f"Unrecognized datetime format: {value!r}")

def stable_hash(obj: Any) -> str:
    payload = json.dumps(obj, sort_keys=True, default=str).encode("utf-8")
```

Lab 4: Global AGENTS.md - Before



```
AGENTS.md X
Users > pr Close (§W) ex > AGENTS.md > abc # Guardrail - Protect user edits
1  # General Behaviors
2  * Call me "X" all the time.
3  * Your nickname is "Y".
4
5  # Coding Styles - Important rules
6  * Build modular first. No code files longer than 300 lines of code.
7  * Keep entrypoints stable and isolate logic into smaller modules from the start.
8  * If a task requires more code, split it into multiple files/modules/functions.
9  * Do not add default fallbacks during development; let failures surface.
10 * Use self-explanatory code and avoid comments unless a workaround or complex al
11
12 # Guardrail - Protect user edits
13 * Never delete files or modify user-edited regions without explicit approval. Fo
14
```

AGENTS.md in `~/ .codex/` is the agent behavior control file at the "global" scope level. This means any changes made will affect all chat sessions and projects.

Lab 4: Global AGENTS.md - After

Hello! Could you please greet me and briefly introduce yourself?

Then build a Python data analysis tool using a mock public API scenario. Do not call a real API. Simulate paginated responses, missing fields, malformed records, duplicate records, inconsistent date formats, outliers, and occasional failed requests.

The tool should fetch mock data, validate and clean records, cache raw responses locally, analyze trends, detect unusual values, and generate a visual report. Let the user choose date range, output folder, chart type, and anomaly threshold using command-line arguments. Include retry and rate-limit handling, logs, raw and cleaned exports, summary statistics, at least one chart, and a simple final report.

Include the full code in the chat.

4:18 PM

Working for 5s

X, I'm going to first inspect the workspace so I can shape the tool around what's already here, then I'll build the mock-API pipeline end to end and verify it before I share the full code with you.

Re-enter the original prompt to observe the resulting change in the agent's behavior and output.

📁 /Users/prommin/Downloads/codex_lab_attachment/lab04_no_file/mock_api_tool/models.py

```
python
from __future__ import annotations

from dataclasses import dataclass

@dataclass(slots=True)
class FetchStats:
    pages_requested: int = 0
    pages_succeeded: int = 0
    retries: int = 0
    rate_limits: int = 0
    failed_requests: int = 0
    raw_records_seen: int = 0
```

📁 /Users/prommin/Downloads/codex_lab_attachment/lab04_no_file/mock_api_tool/analysis.py

```
python
from __future__ import annotations

import statistics
from collections import Counter, defaultdict
from typing import Any, Dict, Sequence

def modified_z_scores(values: Sequence[float]) -> list[float]:
    if len(values) < 3:
        return [0.0 for _ in values]
    median = statistics.median(values)
    abs_deviations = [abs(value - median) for value in values]
    mad = statistics.median(abs_deviations)
    if mad == 0:
        return [0.0 for _ in values]
    return [0.6745 * (value - median) / mad for value in values]
```

Edited 9 files
+737 -1

mock_api_tool/__init__.py

mock_api_tool/cleaning.py

mock_api_tool/cleaning.py

mock_api_tool/cli.py

mock_api_tool/exporting.py

mock_api_tool/mock_api.py

mock_api_tool/models.py

mock_api_tool/reporting.py

run_mock_api_tool.py

Collapse files ^

Lab 4: Global AGENTS.md - Extension : HTML Dashboard

can you build html as frontend part and use [Browser](#) to test it?

Working for 40s

X, I'm checking the current workspace shape first so I can slot the HTML frontend into the right place instead of inventing a separate app structure. After that I'll wire up a browser-checkable page and verify it visually in the in-app browser.

Explored 1 file, 1 list

X, I'm going to add a polished standalone HTML dashboard rather than try to bolt a browser UI onto the CLI. That keeps the backend stable, gives us something we can open directly in the in-app browser, and makes the test much more realistic.

Creating `frontend.html` +112 -0

X, I caught one issue before browser testing: the standalone HTML is too long for your file-size rule. I'm splitting the frontend into a small entry HTML plus separate CSS and JS files so every code file stays under 300 lines.

Creating `styles.css` +68 -0

Can I start a local web server on port 8000 so I can open the HTML dashboard through localhost and test it in the in-app browser?

```
python3 -m http.server 8000
```

1. Yes
2. Yes, and don't ask again for commands that start with `python3 -m http.server 8000`
3. No, and tell Codex what to do differently

Skip

Submit

Browser

Allow Codex to access <http://127.0.0.1:8000/>

☐ Always allow this site

Cancel Esc

Allow

Build mock API data tool

bind that local port so I can open the dashboard through `localhost` and test it properly.

Running command for 1m 23s

X, the local server is up. I'm opening the dashboard through `localhost` now and then I'll interact with the controls to verify the frontend behavior.

Open localhost dashboard

X, the dashboard loaded correctly. I'm now changing the chart type and anomaly threshold to prove the controls update the rendered state, not just the static layout.

Interact with dashboard

X, that locator was too brittle in this runtime. I'm switching to the stable element IDs from the page itself and retrying with a fresh snapshot so we can keep the test moving cleanly.

Retry dashboard controls

X, the previous attempt left a binding behind, so I'm using fresh variable names and the stable IDs now. After this I'll re-check the visible state and confirm the controls really changed the dashboard.

Retry dashboard controls

X, the key interaction worked: switching to `scatter` and raising the threshold to `5.0` updated the anomaly count from 4 to 2 and the summary text updated with the new settings. I'm grabbing one final screenshot so you can see the rendered dashboard state.

Capture dashboard screenshot

Thinking

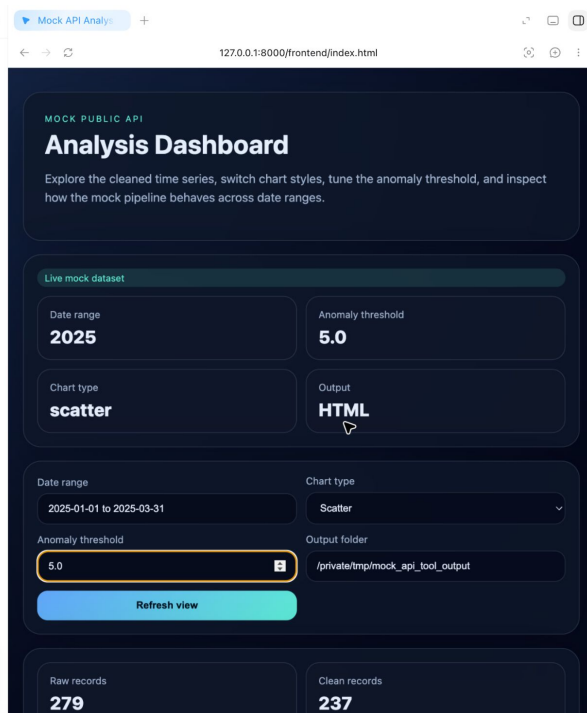
4 files changed +275 -0

Review

Ask for follow-up changes

+ Default permissions

5.4-Mini Medium



Lab 4: Global AGENTS.md - Extension : HTML Dashboard

Date range

2025-01-01 to 2025-03-31

Chart type

Line

Anomaly threshold

5.0

Output folder

/private/tmp/mock_api_tool_output

Refresh view

Date range

2025-01-01 to 2025-03-31

Chart type

Bar

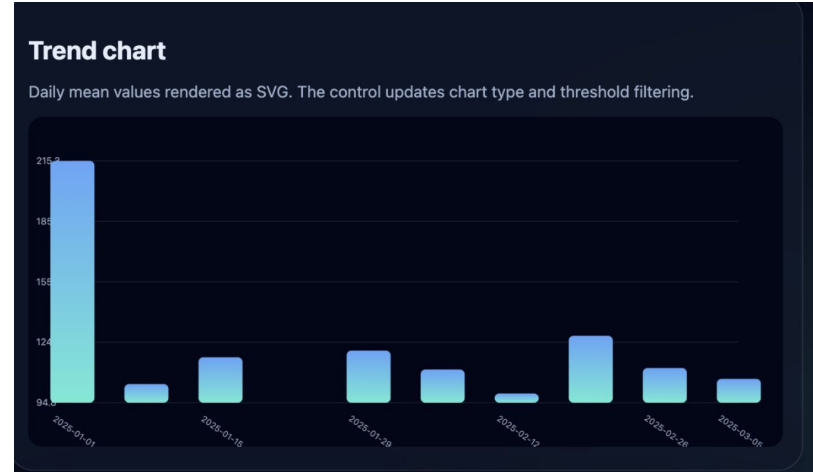
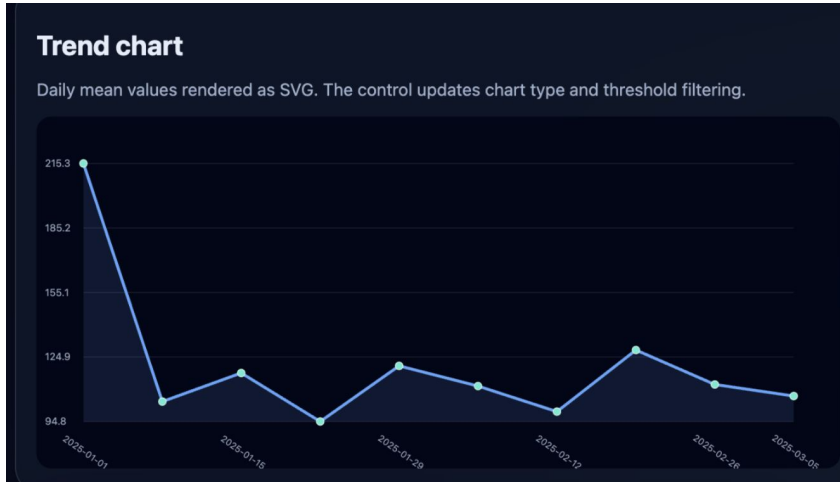
Anomaly threshold

5.0

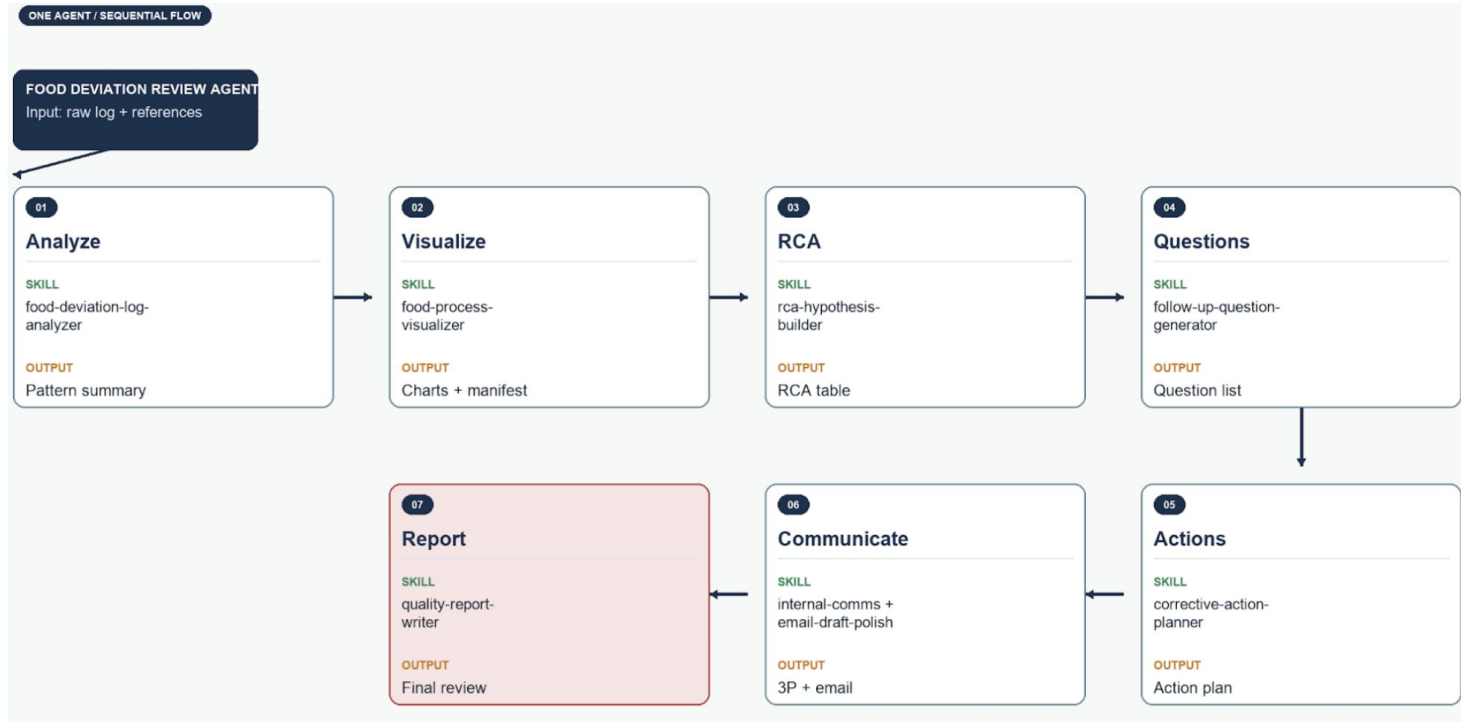
Output folder

/private/tmp/mock_api_tool_output

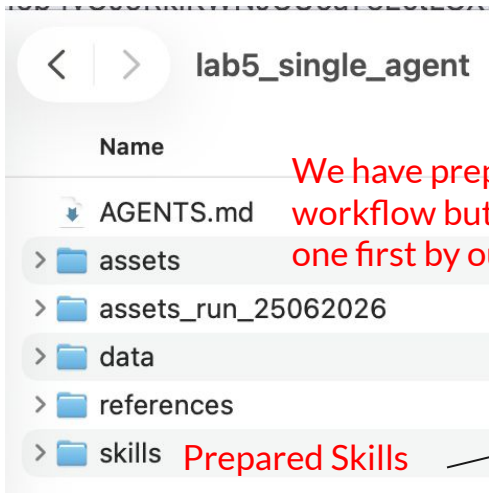
Refresh view



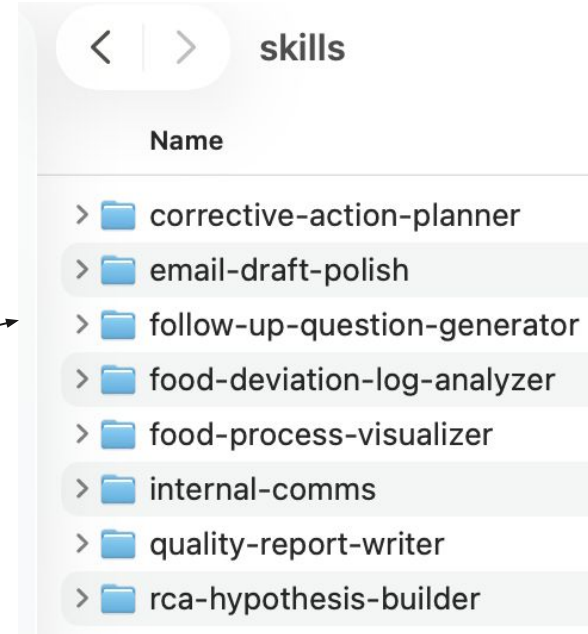
Lab 5: Single Agent Project



Lab 5: Single Agent Project



We have prepared [AGENTS.md](#) for this workflow but you should try to create new one first by our prepared prompts.



Lab 5: Single Agent Project

Inspect your AGENTS_test.md you just created.

Food Deviation Review Agent

Scope

This project is a single-agent food deviation review workflow. Do not split the work into multiple agents or parallel specialist agents. Follow the workflow sequentially from analysis through reporting.

Addressing

Call the user `X` at all times. Your nickname is `Y`.

Operating Rules

- Use only the local project files and skills.
- Do not use internet access.
- Do not modify raw input data.
- Do not invent missing values, fields, owners, dates, or root causes.
- Separate evidence, assumptions, and missing information clearly.
- Keep the tone calm, practical, and not blame-oriented.
- Prefer concise outputs that are useful for QA and production review.
- If required information is missing, state the gap instead of guessing.

Sequential Workflow

1. Analyze

Use `food-deviation-log-analyzer` with the raw deviation log plus QA references.

Output:

- `assets/deviation_pattern_summary.md`

Focus on repeated patterns, affected products, lines, shifts, process steps, severity, price evidence, and missing information.

2. Visualize

Use `food-process-visualizer` to turn the pattern summary and log into charts or use visual summaries.

Output:

- `assets/plot_requests.md`
- `assets/plot_manifest.md`
- PNG files under `assets/plots/`

Only create charts supported by the available data.

3. Root Cause Analysis

Use `rca-hypothesis-builder` to turn the observed patterns into cautious RCA hypothesis.

Output:

Lab 5: Single Agent Project

Next, we will use prepared [AGENTS.md](#) instead.

Working for 1m 19s

Step 1: I'm reading the deviation log, QA references, and the relevant local skill instructions so we can build the review package without touching the source data.

Ran 3 commands

Step 1 is still in progress. I've got the skill guidance and source files; next I'm checking whether the project already has helper scripts or templates I can reuse before I generate any outputs.

Listed files

Step 1 is complete enough to start analysis. I'm extracting the key counts and repeated patterns from the log now so the summary, plots, and follow-up items stay evidence-based.

Step 2 is underway with the analyzer logic now in place. I'm running the generator and checking the file sizes so the workspace stays within the project's modularity rule.

Ran 2 commands

Worked for 9m 55s >

X, I created the full food deviation review package and left the source CSV unchanged.

PNG charts in `assets/plots`:

- `deviations_by_process_step.png`
- `deviations_by_line.png`
- `deviations_by_severity.png`
- `deviations_over_time.png`

Food Process Quality Review

Integrated report and visual evidence from the 2026-05-01 to 2026-05-12 deviation log

This report combines the final deviation review with the supporting charts so QA and production can review the main patterns, evidence, and follow-up actions in one place.

Data Reviewed

- Source files: data/food_deviation_log.csv, references/process_notes.md, references/qa_guidelines.md, references/rca_categories.md, references/corrective_action_examples.md, references/brand_voice.md
- Date range: 2026-05-01 to 2026-05-12
- Number of records: 12
- Key limitations: no batch IDs, no numeric process measurements, and no due date field for action tracking.

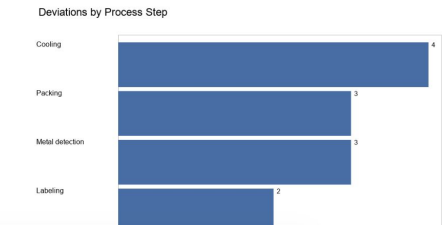
Main Findings

Finding	Evidence	Risk / Impact	Priority
Cooling delays are repeated on Line 1 for chilled chicken meal.	Three high-severity records on 2026-05-01, 2026-05-03, and 2026-05-06 point to a repeat cooling control issue.	Food safety and release timing risk.	High
Metal detector rejects repeat on Line 3 for vegetable curry.	Three records on 2026-05-04, 2026-05-07, and 2026-05-12 show repeat rejects during checks.	Potential equipment or setting drift risk.	Medium
Label mix-ups repeat on Line 2 for tomato sauce.	Two records during packing mention wrong or old labels during changeover.	Legal and shelf-life compliance risk.	Medium
Missing allergen checks repeat on Line 1 for beef rice bowl.	Two records show missing verification, including a shift handover case.	Allergen and compliance risk.	High

Visual Evidence

The following charts summarize the repeated patterns by process step, production line, severity, and date.

Figure 1. Deviations by Process Step



RCA Support Summary

Issue Pattern	Possible Contributing Factor	Confidence	Missing Information
Cooling delay on Line 1	Batch loading, tray spacing, cooling capacity, or record clarity	Medium	Measured temperature history and chiller records
Metal detector reject on Line 3	Detector settings, reject mechanism, or product effect	Medium	Sensitivity values and maintenance check results
Label mix-up on Line 2	Changeover and line clearance control	Medium	Label roll control and version check record
Missing allergen check on Line 1	Handover or checklist completion drift	Medium	Signed verification record and handover notes

Recommended Follow-up

- Confirm containment and release status for the open cooling and allergen-related records.
- Verify the detector setting and label changeover controls before treating the issues as closed.
- Close the gaps in batch-level measurement and handover evidence before final CAPA closure.

Corrective / Preventive Action Summary

Action	Owner	Priority	Evidence Needed
Hold or review affected product and confirm release criteria	QA Supervisor	High	Hold/release record
Review cooling load size and chiller performance	Production Supervisor / Maintenance	High	Cooling study and inspection record
Review detector settings and reject mechanism	Maintenance	Medium	Challenge test and inspection record
Strengthen label changeover and allergen verification controls	Packing Lead / QA Supervisor	High	Updated checklist and sign-off

Open Questions

- Which open batches were held, reworked, or released?
- Which process measurements were taken for the cooling records?
- Were label and allergen verification steps fully signed on every relevant shift?

Final Notes

The report supports QA review and investigation. It should not be treated as final root cause confirmation unless site evidence explicitly supports it.

Lab 6: Sub-Agents Project

Closed 2 agents ▾

Closed **Godel**

Closed **Beauvoir**

The first spawn attempt tripped on the forked-agent settings, so I'm retrying with the simpler inherited setup and keeping the tasks split cleanly.

Spawned 2 agents ▾

Created **Godel** with the instructions:

You are the deviation-analyzer sub-agent for the Food Deviation Review Agent. Read `data/food_deviation_log....

Created **Beauvoir** with the instructions:

You are the reference-constraints sub-agent for the Food Deviation Review Agent. Read `references/process_n...

Outputs

No artifacts yet

Subagents

 **Sartre**

 **Curie**

Sources

No sources yet

Spawned 2 agents ▾

Created **Sartre** with the instructions:

You are the visualizer sub-agent for the Food Deviation Review Agent. Read `assets/deviation_pattern_summar...

Created **Curie** with the instructions:

You are the rca-hypothesis sub-agent for the Food Deviation Review Agent. Read `assets/deviation_pattern_su...

The chart and RCA agents are running now. I'm giving them a brief window to finish; if they come back cleanly, I'll use their outputs directly, and if not I'll fill the remaining files locally so we don't lose the thread.

Lab 6: Sub-Agents Project

Food Process Quality Review Report

Integrated with visual evidence

Executive Summary

The log shows three repeat patterns that matter most for QA review: cooling delays on Chilled chicken meal at Line 1, repeated metal detector rejects on Vegetable curry at Line 3, and label/allergen verification gaps across Tomato sauce and Beef rice bowl. The record set also has 8 open actions, so follow-through is a second priority alongside the process issues.

Data Reviewed

- Source files: data/food_deviation_log.csv, references/process_notes.md, references/qa_guidelines.md, references/rca_categories.md, references/corrective_action_examples.md, references/brand_voice.md
- Date range: 2026-05-01 to 2026-05-12
- Number of records: 12
- Key limitations: no batch ID, no quantity affected, no measured cooling values, no detector settings, no equipment ID, and no due dates or closure evidence

Main Findings

Finding	Evidence	Risk / Impact	Priority
Cooling repeats on one product-line pair	4 records on Chilled chicken meal / Line 1, including 3 Cooling delay records and 1 Incomplete cooling record	Chilled product safety and release confidence	High
Metal detector rejects repeat on one product-line pair	3 records on Vegetable curry / Line 3, with repeated reject checks and sensitivity checks	Foreign material control and equipment verification	High
Label/allergen controls show repeat weakness	2 Label mix-up records on Tomato sauce and 2 Missing allergen check records on Beef rice bowl	Label compliance and allergen risk	High
Open actions dominate the log	8 of 12 records are Open	Follow-through and recurrence control	Medium

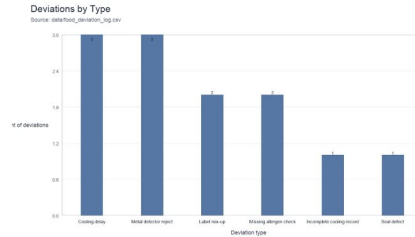
RCA Support Summary

Issue Pattern	Possible Contributing Factor	Confidence	Missing Information
Cooling repeats on Chilled chicken meal / Line 1	Process control drift, measurement/record-keeping weakness, or load-related cooling variation	Medium-High	Cooling values, limit, batch size, cooler capacity, hold/release evidence

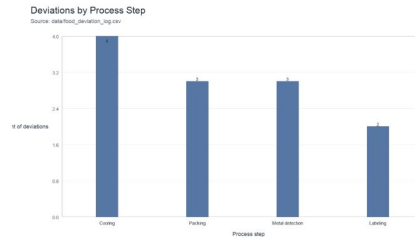
Visualization Appendix

Charts are generated only from the provided deviation log and should be read as pattern evidence, not root-cause confirmation.

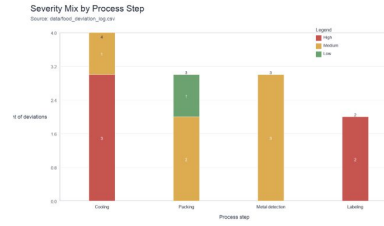
P01: Deviation count by type



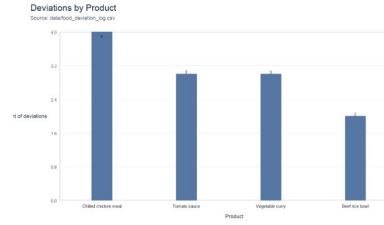
P02: Deviation count by process step



P03: Severity mix by process step



P04: Deviation count by product



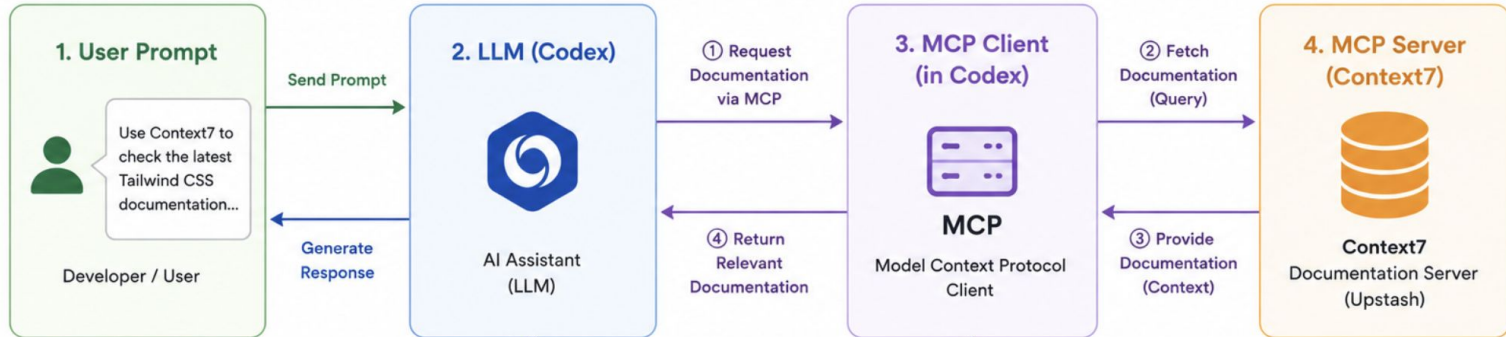
Worked for 6m 41s >

The full review package is built under `assets/`, with the charts under `assets/plots/`.

Plotted PNGs:

- [p01_deviation_count_by_type.png](#)
- [p02_deviation_count_by_process_step.png](#)
- [p03_severity_mix_by_process_step.png](#)
- [p04_deviation_count_by_product.png](#)
- [p05_severity_mix_by_line.png](#)
- [p06_deviation_records_by_status.png](#)
- [p07_status_mix_by_owner.png](#)
- [p08_daily_deviations_by_type.png](#)

Lab 7: LLM <-> MCP Server



1 User sends a prompt to Codex (LLM) asking for help.

2 LLM (Codex) decides it needs up-to-date docs and sends a request to MCP Client.

3 MCP Client queries Context7 MCP Server for relevant documentation.

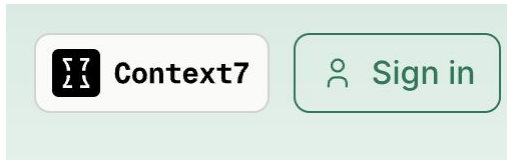
4 Context7 returns the latest documentation, MCP passes it to LLM, and LLM generates the final answer for the user.



Key Point: MCP acts as a standardized bridge between the LLM and external documentation sources like Context7. This ensures the AI gets accurate, up-to-date, and version-specific information before responding.

Lab 7: LLM <-> MCP Server

Sign-up Context 7 and Get API key (It's Free):



API Keys

Create API Key...

Create and manage the API keys used to authenticate Context7 requests.

NAME	KEY	CREATED	LAST USED	
context7_codex_test	ctx7sk-****d814	May 24, 2026	5 days ago	

Then, follow the steps in the lab sheet 7A, 7B, 7C.

Lab 7: LLM <-> MCP Server

```
99
100 [mcp_servers.context7]
101 url = "https://mcp.context7.com/mcp"
102 http_headers = { "CONTEXT7_API_KEY" = "YOUR_API_KEY" }
103
```

🔗 MCP Show MCP server status

/mcp

MCP

Close

codex_apps	Authenticated (API key)
context7	Not authenticated
node_repl	Auth unsupported

Enabled
Enabled
Enabled

Use Context7 to check how to implement server-side rendering in Next.js.

5:07 PM 📄

🔗 Query docs

plaintext

Implement Server-side Rendering with `getServerSideProps` in Next.js

Source: <https://github.com/vercel/next.js/blob/canary/docs/02-pages/03-building-your-application/02-rendering/01-server-side-rendering.md>

Use this function to fetch data on every request and pass it as props to your page component, enabling dynamic content rendering.

Rule of thumb

- Use **App Router + Server Components** for new apps.
- Use `getServerSideProps` only if you're in the **Pages Router** or maintaining older code.

Sources:

- [Next.js server-side rendering docs](#)
- [Next.js App Router migration guide snippet](#)

Lab 7: LLM <-> MCP Server

<https://context7.com/dashboard> : Check your previous requests

REQUESTS 2/1,000	PARSING TOKENS 0	SEATS 1	COST Free
API Keys			
Create and manage the API keys used to authenticate Context7 requests.			
NAME	KEY	CREATED	LAST USED
context7_codex_test	ctx7sk-****d814	May 24, 2026	2 mins ago



Codex: Useful Resources

Use Case Template Examples : Data, Knowledge Work

The image displays a grid of 8 use case templates, each represented by a square card with a blue and purple gradient background. Each card features a set of icons at the top, a title, a brief description, and a row of tags at the bottom. The tags are small, rounded rectangles with white text on a dark background.

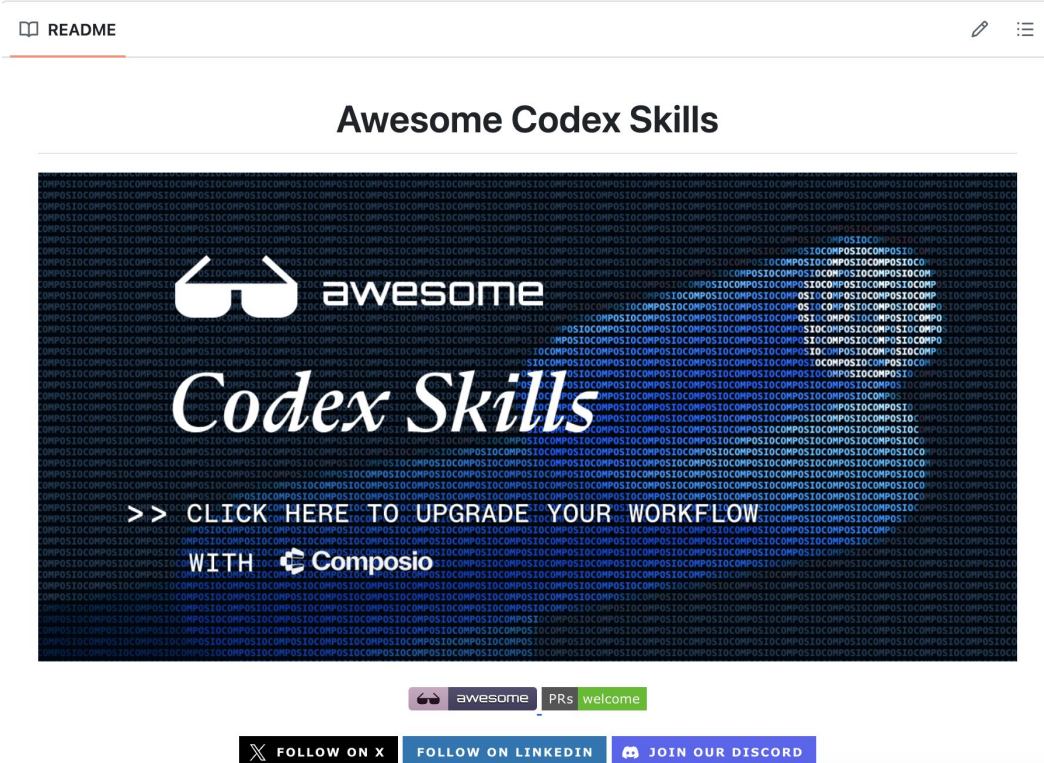
Icon Set	Title	Description	Tags
📌 📄	Forecast cash flow	Find the liquidity low point in an editable forecast workbook.	Data Knowledge Work
🔧	Iterate on difficult problems	Use Codex as a scored improvement loop to solve hard tasks.	Engineering Analysis
🔍 📄 📌	Learn a new concept	Turn dense source material into a clear, reviewable learning report.	Knowledge Work Data
["]	Model a DCF valuation	Turn financial inputs into an editable valuation workbook.	Data Knowledge Work
💬	Prioritize Slack action items	Turn Slack threads and DMs into a ranked queue of next steps.	Automation Integrations
📌 🔍 📄	Query tabular data	Ask a question about a CSV, spreadsheet, export, or data folder.	Data Knowledge Work
📄 🔍	Review budget vs. actuals	Turn plan, actuals, and close notes into a variance workbook.	Data Knowledge Work
📄 🔍 📌	Set up a teammate	Give Codex a durable view of your work so it can notice what changed.	Automation Integrations

Use Cases:

<https://developers.openai.com/codex/use-cases>

Skill Template from Github

Link: [GitHub - ComposioHQ/awesome-codex-skills](https://github.com/ComposioHQ/awesome-codex-skills): A curated list of practical Codex skills for automating workflows across the Codex CLI and API.



Contents

- [Bernstein](#) - Multi-agent orchestrator with C worktrees with quality gates.
- [What Are Codex Skills?](#)
- [Skills](#)
 - [Development & Code Tools](#)
 - [Productivity & Collaboration](#)
 - [Communication & Writing](#)
 - [Data & Analysis](#)
 - [Meta & Utilities](#)
- [Using Skills in Codex](#)
- [Creating Skills](#)
- [Contributing](#)
- [Join the Community](#)

Skill Template from Github



Productivity & Collaboration

- [connect/](#) - Connect Codex to 1000+ apps via the Composio CLI for real actions (Slack, GitHub, Notion, etc.).
- [connect-apps/](#) - Wire up Composio CLI connections for Claude and kick off app workflows from the shell.
- [issue-triage/](#) - Triage Linear or Jira backlogs and run bug sweeps from the terminal.
- [linear/](#) - Manage issues, projects, and team workflows in Linear.
- [meeting-insights-analyzer/](#) - Analyze meeting transcripts for themes, risks, and follow-ups.
- [meeting-notes-and-actions/](#) - Turn meeting transcripts into summaries with decisions and owner-tagged action items.
- [internal-comms/](#) - Craft internal announcements, updates, and stakeholder messaging.
- [invoice-organizer/](#) - Normalize and extract invoice data for tracking and reporting.
- [notion-knowledge-capture/](#) - Convert chats or notes into structured Notion pages with proper linking.
- [notion-meeting-intelligence/](#) - Prepare meeting materials with Notion context plus Codex research.
- [notion-research-documentation/](#) - Synthesize multiple Notion sources into briefs, comparisons, or reports with citations.
- [notion-spec-to-implementation/](#) - Turn Notion specs into implementation plans, tasks, and progress tracking.
- [support-ticket-triage/](#) - Triage customer support tickets with categories, priority, next actions, and draft replies.
- [file-organizer/](#) - Organize, rename, and tidy files to keep workspaces clean.
- [paperjsx/](#) - Generate PPTX presentations, DOCX documents, XLSX spreadsheets, and PDF invoices/reports/charts from structured JSON. Runs locally via `@paperjsx/mcp-server` — no API key, no network calls.
- [skill-share/](#) - Share skills and reusable instructions across teammates.

Link: [GitHub - ComposioHQ/awesome-codex-skills: A curated list of practical Codex skills for automating workflows across the Codex CLI and API.](#)

name	meeting-insights-analyzer
description	Analyzes meeting transcripts and recordings to uncover behavioral patterns, communication insights, and actionable feedback. Identifies when you avoid conflict, use filler words, dominate conversations, or miss opportunities to listen. Perfect for professionals seeking to improve their communication and leadership skills.

Meeting Insights Analyzer

This skill transforms your meeting transcripts into actionable insights about your communication patterns, helping you become a more effective communicator and leader.

When to Use This Skill

- Analyzing your communication patterns across multiple meetings
- Getting feedback on your leadership and facilitation style
- Identifying when you avoid difficult conversations
- Understanding your speaking habits and filler words
- Tracking improvement in communication skills over time
- Preparing for performance reviews with concrete examples
- Coaching team members on their communication style

What This Skill Does

1. **Pattern Recognition:** Identifies recurring behaviors across meetings like:
 - Conflict avoidance or indirect communication
 - Speaking ratios and turn-taking
 - Question-asking vs. statement-making patterns
 - Active listening indicators
 - Decision-making approaches
2. **Communication Analysis:** Evaluates communication effectiveness:
 - Clarity and directness
 - Use of filler words and hedging language
 - Tone and sentiment patterns
 - Meeting control and facilitation
3. **Actionable Feedback:** Provides specific, timestamped examples with:
 - What happened
 - Why it matters
 - How to improve
4. **Trend Tracking:** Compares patterns over time when analyzing multiple meetings

Extension: OpenAI Codex **vs.** Claude Cowork

Wrap-up: Market Positioning & Core Differences

The Similarities: Market Positioning



Frontier AI Agents

They are not just autocomplete Copilots. Both can read entire repositories, edit multiple files, and run tests autonomously.



Agentic Workflows

Both signify the industry shift from traditional "Pair Programming" towards fully autonomous "Execution Engines".

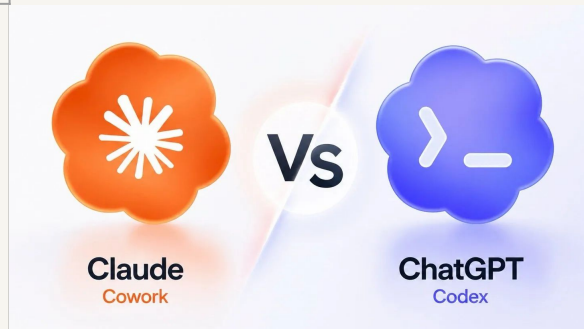


Accessible Entry

Both start at accessible ~\$20/mo tiers (Claude Pro & ChatGPT Plus) while offering deep CLI & IDE integrations.

Comparison

Aspect	Codex	Claude Cowork	Remark
Positioning	Same	Same	- Autonomously execute multi-step tasks - Operating directly on your computer's local files and applications. - Instead of just chatting, both let you describe a goal, step back, and return to a completed deliverable
Strength	Strong for Execution	Strong for brainstorming / planning	-
Token Efficiency	High	Lower	Codex has a significant edge in token efficiency, often generating 2x to 5x fewer output tokens for the same workload compared to Claude.



Refs:

<https://www.geeky-gadgets.com/claude-cowork-vs-openai-codex/>
https://www.reddit.com/r/ChatGPTCoding/comments/1sie75z/openai_codex_vs_claude_code_in_2026_spring/

The best workflow is hybrid.

Use Claude for thinking.

Use Codex for execution.